

gods_eye_v1_approach_2 — Multi-Agent Message Bus Architecture

Title

gods_eye_v1_approach_2: Role-Separated Agents with Typed Message Bus and Persistent Vector Memory

Design Period: Conceptual Iteration 2

Status: Deprecated — Superseded by Approach 3

Predecessor: Approach 1 (Monolithic Sequential Chain)

Problem Statement

Approach 1 demonstrated that a monolithic ReAct loop cannot sustain coherent state across a long penetration testing engagement. The context degrades, findings are lost, and the single-threaded execution model makes any blocking command catastrophic. The problem in Approach 1 was not the LLM's reasoning quality — it was that **the architecture had no separation between the agent that reasons about what to do and the infrastructure that maintains what has been done.**

Approach 2 reframes the problem at a higher level: **how do you coordinate multiple specialized agents that each own a domain of responsibility, share findings across agents, and maintain execution state that survives individual agent failures?**

The answer in this iteration is a **typed message bus** — agents communicate by emitting and consuming structured message events via a shared asyncio queue system. A dedicated PlannerAgent decomposes the goal; a dedicated ExecutorAgent runs commands; a dedicated MemoryAgent writes findings to persistent storage (a Chroma vector DB). The agents are isolated by role, and all cross-agent communication is explicit and typed.

The structural flaw of this approach is not the multi-agent concept — that is sound. The flaw is that **the message bus itself becomes the central coordination bottleneck**: every agent's progress depends on messages being correctly emitted, correctly routed, correctly consumed, and correctly acknowledged. When any of these steps fails — due to message ordering, partial consumption, or agent crashes that leave messages unacknowledged — the system enters inconsistent states that are extremely difficult to detect and recover from.

Additionally, this architecture introduces the OpenAI Swarm library for agent-to-agent handoff orchestration, which adds a second coordination layer on top of the message bus, creating two competing coordination mechanisms with no clear authority boundary.

Proposed Architecture

Overview

Four agents are defined, each as a `LangChain RunnableWithMessageHistory` or a custom agent class wrapping an LLM with a specific tool set:

1. **PlannerAgent** — Receives user goal, decomposes into ordered task list, emits `TASK_ASSIGN` messages.
2. **ExecutorAgent** — Consumes `TASK_ASSIGN` messages, executes shell commands, emits `TASK_RESULT` messages.
3. **MemoryAgent** — Consumes all messages, extracts findings, writes to Chroma vector DB, emits `FINDING_STORED` events.
4. **ReporterAgent** — Consumes `FINDING_STORED` events, maintains a live report buffer, answers user queries.

All agents share access to a central `asyncio.Queue`-based message bus. A `Dispatcher` class routes messages by `MsgType` enum to the appropriate agent's consumer coroutine.

OpenAI Swarm is used for PlannerAgent → ExecutorAgent handoff: the Planner uses Swarm's `handoff_to_agent` primitive to transfer execution context when dispatching a task, so the Executor receives not just a task description but the full planning context.

Component Breakdown

Message Bus (`asyncio.Queue` per `MsgType`) The coordination backbone. Each `MsgType` enum value (`TASK_ASSIGN`, `TASK_RESULT`, `FINDING`, `FINDING_STORED`, `STATUS_UPDATE`) has a dedicated `asyncio` queue. The `Dispatcher` class maintains a mapping of `MsgType → Queue` and provides `emit(msg_type, payload)` and `consume(msg_type)` methods. All agents interact with the bus exclusively through the `Dispatcher`.

Design rationale: Typed queues prevent agents from consuming messages intended for other agents. A `TASK_RESULT` message is only visible to the `MemoryAgent`'s consumer loop, not to the `PlannerAgent`. This enforces unidirectional data flow.

OpenAI Swarm (swarm library) Used for PlannerAgent → ExecutorAgent context handoff. When the Planner decomposes a goal into tasks, it uses Swarm's `handoff_to_agent` function to pass execution context (goal, prior findings, current task) to the Executor. This allows the Executor to inherit the Planner's reasoning context rather than receiving only a task string.

Selection rationale: Swarm provides a lightweight multi-agent orchestration primitive that is LLM-agnostic (uses the OpenAI function-calling format but works with any compatible backend). Its handoff mechanism is simpler than LangGraph's state machine approach for this stage of the design.

LangChain (`RunnableWithMessageHistory`, structured tool calling) Each agent uses LangChain's `RunnableWithMessageHistory` to maintain per-agent conversation history. The

Executor's conversation history is scoped to the current task — it is reset on each new `TASK_ASSIGN` message. The Planner's history is persistent for the session duration.

Chroma (chromadb) Vector database for persistent findings storage. The MemoryAgent writes every finding as a Chroma document with metadata fields: `target`, `task_id`, `severity`, `cve_id` (if applicable), `timestamp`. The Executor and Planner can query Chroma for semantically similar past findings before starting a new task, to avoid re-enumerating known services.

Selection rationale: Chroma is chosen over Qdrant at this stage because it runs fully in-process (no separate server), lowering deployment complexity. The `chromadb` Python package is embedded directly in the agent process.

Limitations: Chroma's in-process model means all agents share a single Chroma client instance with no write isolation. Concurrent writes from MemoryAgent and reads from ExecutorAgent can cause lock contention. Chroma also has no native deduplication — the same CVE written twice creates two documents with identical content, inflating retrieval results with duplicates.

subprocess + Docker SDK (docker-py) ExecutorAgent uses `subprocess.run` for direct host execution and the Docker SDK (`docker-py`) for containerized command execution. The Docker SDK allows the Executor to spin up a container (e.g., a Kali Linux image), execute a command inside it, stream output back, and destroy the container — providing a degree of execution isolation absent in Approach 1.

Limitations: Container startup latency (2-5 seconds per command for cold-start containers) makes per-command containerization impractical for iterative workflows. A persistent container-per-task model is used instead, but this requires explicit container lifecycle management that the architecture does not fully specify.

SearxNG + custom ExploitDB scraper Two search tools: SearxNG (general CVE/technique search) and a custom Python scraper against a locally deployed ExploitDB mirror. The scraper uses `requests` + `BeautifulSoup4` to parse ExploitDB search results and extract PoC commands. This is the first iteration to integrate ExploitDB directly rather than relying on SearxNG to surface exploit information.

Limitations: The ExploitDB scraper is brittle against HTML structure changes. More significantly, the scraper returns raw exploit text (shell scripts, Metasploit modules) that the LLM must interpret — there is no structured extraction of key fields (CVE ID, affected version, payload command).

Failure Points

1. Message Ordering and Partial Consumption Create Inconsistent State

The most critical architectural failure. The message bus guarantees delivery within a single asyncio event loop, but it does not guarantee ordering across agents with different processing speeds. Consider the following sequence:

- PlannerAgent emits `TASK_ASSIGN(task_id=t3)` at t=0.
- ExecutorAgent emits `TASK_RESULT(task_id=t3, status=success)` at t=10.
- MemoryAgent is backlogged processing t=0..10 outputs. It consumes `TASK_RESULT(t3)` at t=15.
- PlannerAgent, having received no `FINDING_STORED` acknowledgment by t=12, re-emits `TASK_ASSIGN(task_id=t3)` (retry logic).
- ExecutorAgent now executes t3 again while MemoryAgent is still processing the first execution's output.

This produces **duplicate task execution with inconsistent finding states**. The Planner does not know whether t3's findings are in Chroma or not. There is no distributed transaction boundary — the message bus has no concept of "acknowledged and persisted" vs "in flight."

2. Swarm Handoff Context Overwrites Executor Conversation History

When PlannerAgent uses `handoff_to_agent` to pass context to ExecutorAgent, Swarm replaces the Executor's active conversation with the handoff context. If the Executor is mid-reasoning on a previous task when a new handoff arrives, the new context silently overwrites the prior state. The Executor has no way to detect this overwrite or recover the interrupted task's reasoning context.

This is a **context collision failure** specific to the Swarm handoff model: it assumes agents are idle when receiving handoffs, but in a concurrent system, agents may be actively reasoning when a handoff arrives.

3. MemoryAgent Becomes a Write Bottleneck

All finding storage is serialized through the MemoryAgent. Every `TASK_RESULT` message must be consumed, processed, and written to Chroma before the corresponding `FINDING_STORED` acknowledgment is emitted. During parallel task execution (even limited parallelism in this design), the MemoryAgent's single consumer loop becomes the bottleneck: Executor emits results faster than MemoryAgent can write them, causing queue depth to grow unboundedly.

More critically: the PlannerAgent's replanning decisions depend on `FINDING_STORED` acknowledgments. If the MemoryAgent is backlogged, the Planner makes replanning decisions based on stale Chroma state — it plans new tasks without knowing about findings that are queued but not yet persisted.

4. Chroma Deduplication Failure Corrupts Retrieval Quality

Chroma has no native upsert-with-deduplication. The MemoryAgent checks for existing documents with a similarity search before writing, but this semantic deduplication is unreliable — two descriptions of the same CVE with different phrasing will pass the similarity threshold and both be written. Over a long engagement, Chroma accumulates duplicate findings that inflate retrieval scores and cause the Executor to treat already-known vulnerabilities as new discoveries.

5. No Approval Gate — Execution Is Still Uncontrolled

Approach 2 inherits Approach 1's most dangerous omission: there is no human approval gate on shell commands. The ExecutorAgent proposes and executes commands in the same step. Adding an approval mechanism was considered but deferred because the message bus model makes blocking for user input architecturally complex — a blocking `input()` call in the Executor's consumer loop would stall the entire asyncio event loop.

This illustrates a deeper problem: **the message bus architecture makes synchronous human-in-the-loop interactions structurally difficult**, because the bus assumes all consumers are non-blocking coroutines.

6. PlannerAgent Has No Access to Real-Time Execution State

The PlannerAgent only receives information via `FINDING_STORED` messages from MemoryAgent. It has no visibility into:

- Commands currently being executed by ExecutorAgent.
- Commands that were attempted and failed (non-zero exit) but did not produce a finding.
- The Executor's current retry count on a stuck task.

This means the Planner cannot make informed replanning decisions until a task fully completes or fails — it cannot intervene mid-task when it can see the Executor is struggling. The message bus's event-driven model creates **planning blindness during execution**.

7. Agent Crash Leaves Messages Unacknowledged

If the ExecutorAgent crashes mid-task (process exception, OOM, Docker SDK failure), the `TASK_ASSIGN` message it was processing is marked as consumed (asyncio Queue `task_done()` was called on consume). The task result is never emitted. The PlannerAgent waits indefinitely for a `FINDING_STORED` acknowledgment that will never arrive. There is no dead-letter queue, no timeout on task completion, and no agent health monitoring. **A single agent crash silently halts the entire system.**

Comparison to Approach 1

Approach 2 solves Approach 1's most acute problems:

Problem in Approach 1	How Approach 2 Addresses It
Single monolithic agent	Four role-separated agents with distinct tool sets
In-process-only memory	Chroma vector DB provides persistent finding storage
No replanning	PlannerAgent can re-emit <code>TASK_ASSIGN</code> on failure messages
Output truncation	MemoryAgent receives full <code>TASK_RESULT</code> and extracts findings with LLM
Context collapse	Per-agent conversation history scoped to role

However, Approach 2 introduces new failure modes that are architecturally more complex than Approach 1's failures: message ordering inconsistencies, Swarm context collisions, MemoryAgent bottlenecks, and crash-induced silent halts are all harder to debug and recover from than a simple context window overflow.

Why This Is NOT the Final PDF Architecture

Approach 2 introduces multi-agent separation and persistent memory — both correct directions — but the message bus coordination model is fundamentally wrong for this use case:

- Messages are not the right coordination primitive for autonomous agents.** Messages imply a sender knows the receiver, knows when the receiver is ready, and can interpret the receiver's state. In a genuinely autonomous system, agents should coordinate through shared state, not point-to-point message passing.
- The approval gate problem is unsolved.** The message bus model makes blocking for user input architecturally incompatible. The PDF solves this with `asyncio.Event` — a blocking primitive that does not stall the event loop.
- Swarm's handoff model assumes serial execution.** The PDF's architecture explicitly supports concurrent Hacker instances. Swarm's handoff model is serial by design — one active agent at a time per handoff chain.
- Chroma's in-process model cannot support multi-agent concurrent writes.** The PDF uses Qdrant (server-based, supports concurrent client connections) precisely to solve this.
- There is still no Analyst-equivalent role.** The MemoryAgent writes findings but does not extract structured nuances (OS, port states, credential discoveries) and does not route structured intelligence to the Planner. The PDF's Analyst is a dedicated intelligence extraction and routing layer that the MemoryAgent cannot replace.

What the PDF Fixes Over This Approach

Failure in Approach 2	PDF Solution
Message ordering inconsistencies	Shared memory (state.json) as single source of truth; no message passing
Swarm context collision on handoff	Supervisor dispatches task context explicitly; Hacker is stateless per task
MemoryAgent write bottleneck	Analyst writes to Mem0/Qdrant asynchronously per output, not per task
Chroma deduplication failure	Analyst deduplicates before writing; same CVE on same host = one entity update
No approval gate	<code>propose_command()</code> with <code>asyncio.Event</code> blocking, architecturally clean in async model
Planning blindness during execution	Analyst routes per-command nuances to Supervisor in real time, not just on task completion
Agent crash leaves silent halt	Analyst failure modes escalate via <code>signal_stuck</code> ; Supervisor has full DAG state visibility
Swarm serial handoff model	Supervisor dispatches concurrently to multiple Hacker coroutines
No structured intelligence extraction	Dedicated Analyst role with typed schema extraction (ports, creds, OS, WAF, etc.)